

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:
«Веб-технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Разработка браузерного расширения»

Задание уровня «Senior» Вариант №2

Выполнил:

Суханкулиев Мухаммет, ИСУ_ID 341992,
студент группы N3246, поток ВЕБ-ТЕХ. N23 1.3



(подпись)

Проверил:

Менщиков Александр Алексеевич,
к.т.н., доцент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Разработка браузерного расширения.....	4
1.1 Структура проекта.....	4
1.2 Описание алгоритма работы.....	4
1.3 Скриншоты.....	5
1.4 Пример JSON	5
Заключение.....	6
Список использованных источников.....	7

ВВЕДЕНИЕ

Цель работы – разработать приложение, которое экспортирует открытые вкладки в JSON.

Описание задания:

Необходимо написать браузерное расширение, которое анализирует открытые вкладки и формирует JSON, пригодный для экспорта (название, URL, метаинформация и др.). У приложения должна быть HTML страница с интерфейсом, содержащая следующие сведения:

- Список сделанных экспортов с отображением даты и возможностью открыть (expand) или скачать JSON;
- Кнопка “импорт”, позволяющая импортировать JSON (должна открывать и закрывать существующие вкладки, восстанавливая состояние вкладок на момент экспорта);
- У каждого состояния из списка должна быть кнопка “откатиться к состоянию”, которая автоматически приводит вкладки в соответствие с этим JSON.

При разработке запрещается использовать любые библиотеки, кроме встроенных браузерных API или сторонних API с реализацией запросов к ним через HTTP.

1 РАЗРАБОТКА БРАУЗЕРНОГО РАСШИРЕНИЯ

1.1 Структура проекта

Разработка велась с использованием Manifest V3 API. Структура:

```
SuhangulyyevM_Senior_2/  
├── manifest.json  
├── options.html # страница с историей экспортов: отображение, скачивание,  
├── options.js    восстановление и удаление  
├── popup.html   # интерфейс управления экспортом и импортом вкладок в  
├── popup.js     компактной форме  
├── utils.js     # бизнес-логика экспорта, восстановления и работы с  
                chrome.storage;  
├── icon.png  
└── styles.css
```

1.2 Описание алгоритма работы

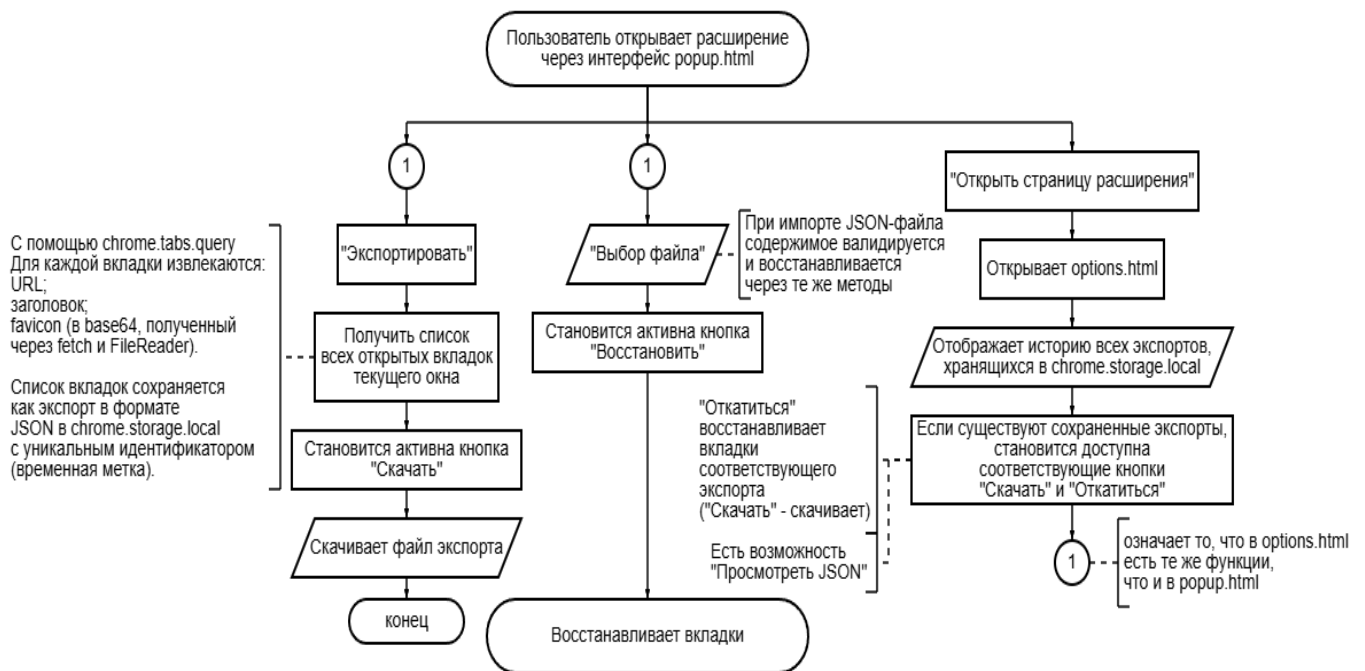


Рисунок 1 – Псевдо-блок-схема работы расширения

Повторно используемая логика (получение вкладок, сериализация, сохранение, восстановление, удаление) вынесена в модуль `utils.js`, который подключается как в `popup.js`, так и в `options.js`.

p.s. Также была добавлена возможность очистить историю.

1.3 Скриншоты

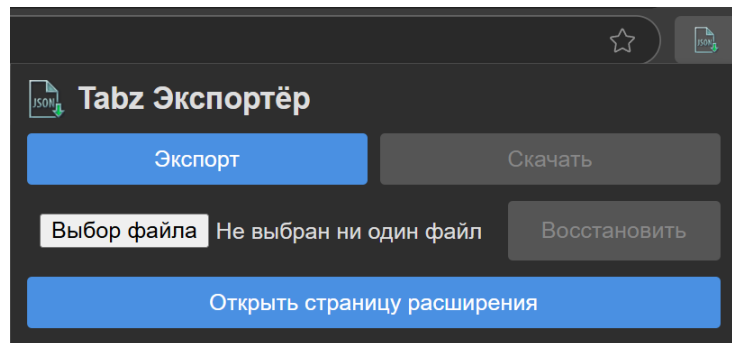


Рисунок 2 – Окно rorup.html

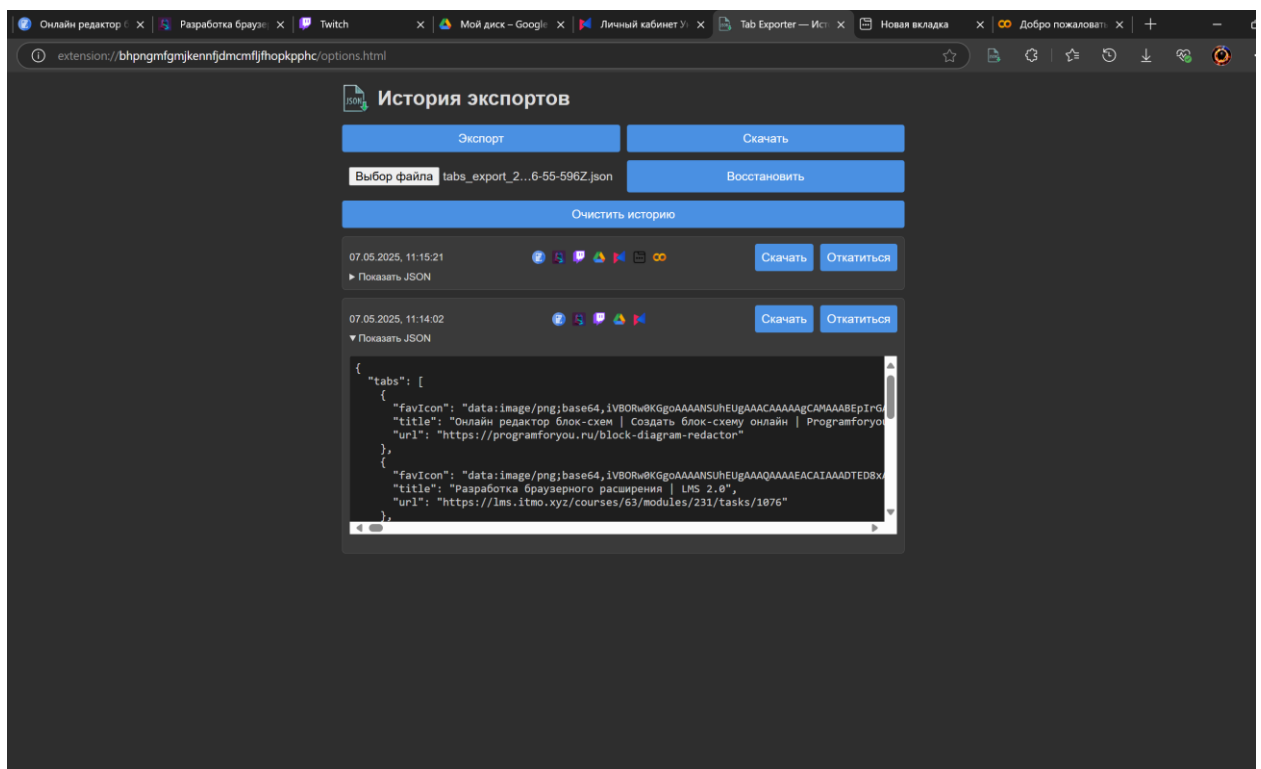


Рисунок 3 – Страница options.html

1.4 Пример JSON

tabs_export_2025-05-07T07-56-55-596Z.json

```
{
  "timestamp": "2025-05-07T07:56:55.596Z",
  "tabs": [
    {
      "title": "Example Domain",
      "url": "https://example.com",
      "favicon": "data:image/png;base64,PHN2ZyB4bWxucz0naHR0cDovL3d3dy5..."
    },
    ...
  ]
}
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было разработано браузерное расширение, реализующее экспорт и импорт открытых вкладок в формате JSON. Расширение обеспечивает возможность сохранения состояния вкладок с метаданными (название, URL, иконка), а также восстановления их при необходимости. Реализован пользовательский интерфейс, включающий всплывающее окно (`popup.html`) для быстрого экспорта и импорта, и страницу настроек (`options.html`) с историей экспортов и функцией восстановления.

При разработке были использованы возможности Manifest V3 и стандартные API браузера Chrome (`chrome.tabs`, `chrome.storage`, `FileReader`, `Blob`). Повторно используемая логика была вынесена в отдельный модуль `utils.js`, что обеспечило модульность.

Работа над проектом позволила закрепить навыки работы с API браузера, манипуляции с вкладками и организации взаимодействия между скриптами и пользовательским интерфейсом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Менщиков А. А. Прогрессивные Веб приложения / Менщиков А. А. [Электронный ресурс] // LMS: [сайт]. — URL: https://lms.itmo.xyz/uploads/fwrg69pf47jno0vf5jwi9dw0b91zxdtp/Лекция_7._Прогрессивные_и_браузерные_Веб_приложения.pdf (дата обращения: 30.04.2025).
2. Менщиков А. А. Лекция 7. Прогрессивные и браузерные Веб приложения / Менщиков А. А. [Электронный ресурс] // GitHub: [сайт]. — URL: https://github.com/cs-itmo/webtech_2024/tree/master/lecture_materials/l7-pwa (дата обращения: 30.04.2025).
3. James Johnson A Simple Progressive Web App Tutorial / James Johnson [Электронный ресурс] // Medium: [сайт]. — URL: <https://medium.com/james-johnson/a-simple-progressive-web-app-tutorial-f9708e5f2605> (дата обращения: 03.05.2025).
4. altrapeznikov Пишем простое расширение для браузера / altrapeznikov [Электронный ресурс] // Хабр: [сайт]. — URL: <https://habr.com/ru/articles/887098/> (дата обращения: 05.05.2025).
5. Bhaskar Sawant How to transfer data between tabs using a Chrome extension / Bhaskar Sawant [Электронный ресурс] // Dev: [сайт]. — URL: https://dev.to/bhaskar_sawant/how-to-transfer-data-between-tabs-using-a-chrome-extension-1ghn (дата обращения: 05.05.2025).

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг А.1 – manifest.json

```
{
  "manifest_version": 3,

  "name": "Tabs Exporter",
  "description": "Экспорт и восстановление открытых вкладок в формате JSON.",
  "version": "1.0",

  "permissions": [
    "tabs",
    "downloads",
    "storage",
    "scripting"
  ],

  "host_permissions": ["<all_urls>"],

  "action": {
    "default_title": "Tabs Exporter",
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icon.png",
      "32": "icon.png",
      "48": "icon.png",
      "128": "icon.png"
    }
  },

  "options_page": "options.html",

  "icons": {
    "16": "icon.png",
    "32": "icon.png",
    "48": "icon.png",
    "128": "icon.png"
  }
}
```

Листинг А.2 – popup.html

```
<!DOCTYPE html>
<html lang="ru">

<head>

  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <title>Tabs Exporter</title>
  <link rel="stylesheet" href="styles.css">

</head>

<body class="popup-body">

  <header class="popup-header">
```

```

    
    <h2 class="popup-title">Tabz Экспортёр</h2>
</header>

<section class="action-buttons">
  <div class="row">
    <button id="exportTabs" class="btn">Экспорт</button>
    <button id="downloadLast" class="btn" disabled>Скачать</button>
  </div>
  <div class="row">
    <input type="file" id="importFile" accept="application/json"
class="file-input">
    <button id="restoreImport" class="btn" disabled>Восстановить</button>
  </div>
</section>

<section class="options-section">
  <button id="openOptions" class="btn full-width">Открыть страницу
расширения</button>
</section>

<script type="module" src="popup.js"></script>

</body>

</html>

```

Листинг А.3 – popup.js

```

import {
  exportCurrentTabs,
  importTabsFromFile,
  restoreTabsFromPayload,
  getLastExport
} from './utils.js';

let lastExportPayload = null;

const elements = {
  exportBtn: document.getElementById('exportTabs'),
  downloadBtn: document.getElementById('downloadLast'),
  importInput: document.getElementById('importFile'),
  restoreBtn: document.getElementById('restoreImport'),
  openOptionsBtn: document.getElementById('openOptions')
};

function handleExport() {
  exportCurrentTabs()
    .then(payload => {
      lastExportPayload = payload;
      elements.downloadBtn.disabled = false;
    })
    .catch(error => {
      console.error('Ошибка при экспорте вкладок:', error);
    });
}

function handleDownload() {
  if (lastExportPayload) {
    window.downloadJson(lastExportPayload);
  }
}

```

```

function handleImport(e) {
    const file = e.target.files[0];
    if (!file) return;

    importTabsFromFile(file)
        .then(payload => {
            lastExportPayload = payload;
            elements.restoreBtn.disabled = false;
        })
        .catch(error => {
            console.error('Ошибка при импорте JSON:', error);
            alert('Неверный JSON-файл');
            e.target.value = '';
        });
}

function handleRestore() {
    getLastExport()
        .then(payload => {
            if (payload && Array.isArray(payload.tabs)) {
                restoreTabsFromPayload(payload).catch(console.error);
            }
        })
        .catch(error => {
            console.error('Ошибка при восстановлении вкладок:', error);
        });
}

function openOptions() {
    chrome.runtime.openOptionsPage();
}

// Обработчики событий
elements.exportBtn.addEventListener('click', handleExport);
elements.downloadBtn.addEventListener('click', handleDownload);
elements.importInput.addEventListener('change', handleImport);
elements.restoreBtn.addEventListener('click', handleRestore);
elements.openOptionsBtn.addEventListener('click', openOptions);

```

Листинг A.4 – options.html

```

<!DOCTYPE html>
<html lang="ru">

<head>

    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title>Tab Exporter – История экспортов</title>
    <link rel="stylesheet" href="styles.css">

</head>

<body class="options-body">

    <header class="options-header">
        
        <h1>История экспортов</h1>
    </header>

```

```

<section class="action-buttons">
  <div class="row">
    <button id="exportTabs" class="btn">Экспорт</button>
    <button id="downloadLast" class="btn" disabled>Скачать</button>
  </div>
  <div class="row">
    <input type="file" id="importFile" accept="application/json"
class="file-input">
    <button id="restoreImport" class="btn" disabled>Восстановить</button>
  </div>
  <div class="row">
    <button id="clearHistory" class="btn danger">Очистить историю</button>
  </div>
</section>

<section class="export-history">
  <div id="exportList">Загрузка...</div>
</section>

<script type="module" src="options.js"></script>

</body>

</html>

```

Листинг A.5 – options.js

```

import {
  exportCurrentTabs,
  importTabsFromFile,
  restoreTabsFromPayload,
  getLastExport,
  getAllExports,
  clearExportHistory
} from './utils.js';

let lastExportPayload = null;

const elements = {
  exportBtn: document.getElementById('exportTabs'),
  downloadBtn: document.getElementById('downloadLast'),
  importInput: document.getElementById('importFile'),
  restoreBtn: document.getElementById('restoreImport'),
  exportList: document.getElementById('exportList'),
  clearBtn: document.getElementById('clearHistory')
};

async function render() {
  elements.exportList.innerHTML = '';

  try {
    const entries = await getAllExports();

    if (!entries.length) {
      elements.exportList.textContent = 'Нет сохранённых экспортов';
      return;
    }
    // Создание элементов списка экспортов
    entries.forEach(([key, payload]) => {
      const item = createExportItem(payload);
      elements.exportList.appendChild(item);
    });
  }
}

```

```

    } catch (error) {
      console.error('Ошибка при рендеринге истории:', error);
    }
  }

function createExportItem(payload) {
  const item = document.createElement('div');
  item.className = 'export-item';

  const header = createExportHeader(payload);
  item.appendChild(header);

  const details = createExportDetails(payload);
  item.appendChild(details);

  return item;
}

function createExportHeader(payload) {
  const header = document.createElement('div');
  header.className = 'export-header';

  // Дата
  const dateEl = document.createElement('span');
  dateEl.className = 'date';
  dateEl.textContent = new Date(payload.timestamp).toLocaleString();
  header.appendChild(dateEl);

  // Иконки вкладок
  const iconContainer = document.createElement('div');
  iconContainer.className = 'icon-container';

  payload.tabs.forEach(tab => {
    if (tab.favIcon) {
      const img = document.createElement('img');
      img.src = tab.favIcon;
      img.alt = tab.title;
      img.className = 'tab-icon';
      iconContainer.appendChild(img);
    }
  });

  header.appendChild(iconContainer);

  // Кнопки действий
  const btns = document.createElement('div');
  btns.className = 'item-buttons';

  const downloadBtn = document.createElement('button');
  downloadBtn.className = 'btn';
  downloadBtn.textContent = 'Скачать';
  downloadBtn.onclick = () => window.downloadJson(payload);
  btns.appendChild(downloadBtn);

  const restoreBtn = document.createElement('button');
  restoreBtn.className = 'btn';
  restoreBtn.textContent = 'Откатиться';
  restoreBtn.onclick = () =>
    restoreTabsFromPayload(payload).catch(console.error);
  btns.appendChild(restoreBtn);

  header.appendChild(btns);

```

```

    return header;
}

function createExportDetails(payload) {
    const details = document.createElement('details');
    const summary = document.createElement('summary');
    summary.textContent = 'Показать JSON';

    const pre = document.createElement('pre');
    pre.textContent = JSON.stringify(payload, null, 2);

    details.append(summary, pre);

    return details;
}

function handleExport() {
    exportCurrentTabs()
        .then(payload => {
            lastExportPayload = payload;
            elements.downloadBtn.disabled = false;
            render();
        })
        .catch(error => {
            console.error('Ошибка при экспорте:', error);
        });
}

function handleDownload() {
    if (lastExportPayload) {
        window.downloadJson(lastExportPayload);
    }
}

function handleImport(e) {
    const file = e.target.files[0];
    if (!file) return;

    importTabsFromFile(file)
        .then(payload => {
            lastExportPayload = payload;
            elements.restoreBtn.disabled = false;
            render();
        })
        .catch(error => {
            console.error('Ошибка при импорте:', error);
            alert('Неверный JSON-файл');
            e.target.value = '';
        });
}

function handleRestore() {
    getLastExport()
        .then(payload => {
            if (payload && Array.isArray(payload.tabs)) {
                restoreTabsFromPayload(payload).catch(console.error);
            }
        })
        .catch(error => {
            console.error('Ошибка при восстановлении вкладок:', error);
        });
}

```

```

}

function handleClearHistory() {
  const confirmClear = confirm('Подтвердите удаление истории');
  if (!confirmClear) return;

  clearExportHistory()
    .then(render)
    .catch(error => {
      console.error('Ошибка при очистке истории:', error);
    });
}

// Обработчики событий
elements.exportBtn.addEventListener('click', handleExport);
elements.downloadBtn.addEventListener('click', handleDownload);
elements.importInput.addEventListener('change', handleImport);
elements.restoreBtn.addEventListener('click', handleRestore);
elements.clearBtn.addEventListener('click', handleClearHistory);

document.addEventListener('DOMContentLoaded', render);

```

Листинг А.6 – utils.js

```

const LAST_EXPORT_KEY = 'last_export';

/**
 * Экспорт текущих вкладок
 */
export async function exportCurrentTabs() {
  const tabs = await chrome.tabs.query({ currentWindow: true });
  const tabsInfo = [];

  for (const t of tabs) {
    let favIcon = '';
    if (t.favIconUrl) {
      try {
        const res = await fetch(t.favIconUrl);
        const blob = await res.blob();
        favIcon = await new Promise(resolve => {
          const reader = new FileReader();
          reader.onloadend = () => resolve(reader.result);
          reader.readAsDataURL(blob);
        });
      } catch (e) {
        console.warn(`Не удалось загрузить favicon ${t.favIconUrl}`, e);
      }
    }
    tabsInfo.push({
      title: t.title,
      url: t.url,
      favIcon
    });
  }

  const payload = {
    timestamp: new Date().toISOString(),
    tabs: tabsInfo
  };

  const key = `export_${payload.timestamp}`;
  await chrome.storage.local.set({

```

```

    [key]: payload,
    [LAST_EXPORT_KEY]: payload
  });

  return payload;
}

/**
 * @param {File} file - Файл для импорта
 */
export async function importTabsFromFile(file) {
  const text = await file.text();
  const json = JSON.parse(text);
  await chrome.storage.local.set({ [LAST_EXPORT_KEY]: json });
  return json;
}

/**
 * @param {{timestamp: string, tabs: Array}} payload - Данные для
восстановления вкладок
 */
export async function restoreTabsFromPayload(payload) {
  const originalTabs = await chrome.tabs.query({ currentWindow: true });
  const originalIds = originalTabs.map(t => t.id);
  const urlsToKeep = payload.tabs.map(t => t.url);

  // Открываем вкладки, которые должны быть восстановлены
  const tabsToOpen = payload.tabs.filter(t => !originalTabs.some(o => o.url
=== t.url));
  for (const tab of tabsToOpen) {
    try {
      await chrome.tabs.create({ url: tab.url });
    } catch (error) {
      console.warn(`Не удалось открыть URL: ${tab.url}`, error);
    }
  }

  // Закрываем все вкладки, которые не находятся в списке для восстановления
  const tabsToClose = originalTabs.filter(t => !urlsToKeep.includes(t.url));
  const tabIdsToClose = tabsToClose.map(t => t.id);

  if (tabIdsToClose.length > 0) {
    await chrome.tabs.remove(tabIdsToClose); // Закрываем все вкладки,
которые не должны быть восстановлены
  }
}

/**
 * Возвращает последний экспорт
 */
export async function getLastExport() {
  const stored = await chrome.storage.local.get(LAST_EXPORT_KEY);
  return stored[LAST_EXPORT_KEY];
}

/**
 * Получает все экспортированные вкладки
 */
export async function getAllExports() {
  const all = await chrome.storage.local.get(null);
  return Object.entries(all)

```

```

        .filter(([k, v]) => k.startsWith('export_') && v &&
Array.isArray(v.tabs))
        .sort((a, b) => new Date(b[1].timestamp) - new Date(a[1].timestamp));
    }

/**
 * Очистка истории экспорта
 */
export async function clearExportHistory() {
    const all = await chrome.storage.local.get(null);
    const keys = Object.keys(all).filter(k => k.startsWith('export_') || k ===
LAST_EXPORT_KEY);
    await chrome.storage.local.remove(keys);
}

/**
 * @param {Array<string>} urls - Список URL для открытия
 * @param {Array<number>} originalIds - Список ID вкладок для закрытия
 */
window.restoreTabs = async function(urls, originalIds) {
    try {
        for (const url of urls) {
            try {
                await chrome.tabs.create({ url });
            } catch (error) {
                console.warn(`Не удалось открыть URL: ${url}`, error);
            }
        }

        for (const id of originalIds) {
            try {
                await chrome.tabs.remove(id);
            } catch (error) {
                console.warn(`Не удалось закрыть вкладку с ID: ${id}`, error);
            }
        }
    } catch (error) {
        console.error('Ошибка при восстановлении вкладок:', error);
    }
};

/**
 * @param {{timestamp: string, tabs: Array}} payload - Данные для экспорта
 */
window.downloadJson = function(payload) {
    try {
        const json = JSON.stringify(payload, null, 2);
        const blob = new Blob([json], { type: 'application/json' });
        const url = URL.createObjectURL(blob);

        chrome.downloads.download({
            url,
            filename: `tabs_export_${payload.timestamp.replace(/[:.]/g, '-')}.json`
        });
    } catch (error) {
        console.error('Ошибка при создании и загрузке JSON:', error);
    }
};

```

Листинг A.7 – styles.css

```
:root {
```

```

    font-size: 16px;
    --color-bg: #2e2e2e;
    --color-fg: #ddd;
    --color-accent: #4a90e2;
    --color-accent-dark: #3a71b2;
    --color-disabled-bg: #555;
    --color-disabled-fg: #888;
    --color-border: #555;
    --color-pre-bg: #1e1e1e;
    --color-pre-fg: #cfcfcf;
    --border-radius: 4px;
    --spacing: 10px;
    --font-main: Arial, sans-serif;
}

html, body {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: var(--font-main);
    background: var(--color-bg);
    color: var(--color-fg);
}

h1, h2 {
    margin: 0;
}

/* Popup */
.popup-body {
    width: 460px;
    padding: var(--spacing);
    max-height: 160vh;
    overflow-y: auto;
}

.popup-header {
    display: flex;
    align-items: center;
    margin-bottom: var(--spacing);
}

.popup-icon {
    width: 24px;
    height: 24px;
    margin-right: 8px;
}

.popup-title {
    font-size: 1.2rem;
}

/* Options */
.options-body {
    background: var(--color-bg);
    color: var(--color-fg);
    padding: 15px;
    max-width: 700px;
    margin: auto;
}

.options-header {

```

```

        display: flex;
        align-items: center;
        margin-bottom: 15px;
    }

    .options-icon {
        width: 32px;
        height: 32px;
        margin-right: 10px;
    }

    /* Layout */
    .row {
        display: flex;
        gap: 8px;
        margin-bottom: var(--spacing);
    }

    .full-width {
        width: 100%;
    }

    /* Buttons */
    .btn {
        flex: 1;
        padding: 0.5rem;
        font-size: 0.9rem;
        background-color: var(--color-accent);
        color: #fff;
        border: none;
        border-radius: 3px;
        cursor: pointer;
        white-space: normal;
        word-break: keep-all;
        text-align: center;
        transition: background-color 0.2s ease-in-out;
    }

    .btn:hover {
        background-color: var(--color-accent-dark);
    }

    .btn.disabled {
        background-color: var(--color-disabled-bg);
        color: var(--color-disabled-fg);
        cursor: default;
    }

    .btn.danger {
        background-color: var(--color-accent);
    }

    .btn.danger:hover {
        background-color: var(--color-accent-dark);
    }

    /* Others */
    .icon-container {
        display: flex;
        flex-wrap: wrap;
        gap: 8px;
        row-gap: 8px;
        max-height: 300px;
    }

```

```

        overflow-y: auto;
        padding: 4px;
        border-radius: var(--border-radius);
        justify-content: center;
    }

    .tab-icon {
        width: 16px;
        height: 16px;
        border-radius: 2px;
        object-fit: cover;
    }

    .file-input {
        flex: 1;
        padding: 0.5rem;
        font-size: 0.9rem;
    }

    /* История экспортов */
    .export-item {
        background: #3a3a3a;
        border: 1px solid var(--color-border);
        margin-bottom: var(--spacing);
        padding: 8px;
        border-radius: var(--border-radius);
    }

    .export-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .item-buttons {
        display: flex;
        gap: 8px;
        flex-wrap: wrap;
    }

    /* JSON */
    .export-item pre {
        background: var(--color-pre-bg);
        color: var(--color-pre-fg);
        border: 1px solid #444;
        padding: 6px;
        overflow-x: auto;
        max-height: 200px;
        font-size: 0.85rem;
        border-radius: 3px;
    }

```